A Hybrid Agentic Architecture for Autonomous LLM-Security Threat-Intelligence Collection over Live Vulnerability Sources

Anonymous Author(s)
Affiliation withheld for review
Contact withheld

***Abstract*—The security of large language model (LLM) applications is a fast-moving target whose evidence is scattered across heterogeneous public feeds: the National Vulnerability Database (NVD), the CISA Known Exploited Vulnerabilities (KEV) catalog, the OSV.dev open-source advisory database, and the arXiv research corpus. Existing collection pipelines query these sources with fixed templates, a fixed source rotation, and a fixed number of rounds, leaving the genuinely hard decisions—*which source to query, what to search for, which advanced operators to use, and when to stop*—hard-coded rather than reasoned about. We present the architecture of an autonomous intelligence-collection agent that delegates exactly these four decisions to an LLM operating inside a native tool-use loop, while a deterministic Python controller retains the round skeleton, budget safety valves, and persistence. The design rests on four ideas: (i) a *hybrid controller* in which every LLM decision is a structured, tool-forced call that falls back to a shared deterministic rule on any failure, so behavior never drops below a verified baseline; (ii) four *typed source tools* exposed through an in-process MCP server that surface each feed's advanced query operators directly to the agent; (iii) a *UCB1 multi-armed bandit* that recommends sources from accumulated per-source yield while leaving the final choice to the agent; and (iv) a *three-tier relevance filter* (rule, embedding, flash-LLM) and a compact context digest that keep the decision loop cheap and auditable. We describe the engine-factory abstraction that lets a deterministic and an agentic engine share one rule implementation, a *coverage-first* termination policy in which a depth-quota coverage gate and a marginal-information stall detector bound the LLM stop decision, and a same-budget A/B evaluation over nine frozen LLM-security collection goals (two of them multi-topic), each engine run three times per goal on live sources. The results reframe *when* autonomy pays. On the primary objective—completeness of target-topic coverage to a depth quota—the agentic engine reaches full coverage on six of nine goals versus four for the well-tuned deterministic baseline, is never lower on any goal, and lifts mean coverage from 0.69 to 0.95 ($+38\%$), closing sparse gaps the fixed policy never reaches (e.g. agent-tool-abuse: 1.00 vs 0.00). This costs per-call efficiency: averaged over goals the agent is 8% lower on relevant-items-per-call because it spends calls chasing sparse topics, while retrieving 60% more items per call and terminating at the hindsight-optimal round on all nine goals (versus a mean one-fifth-round overshoot for the baseline), at 97.7% live structured-decision reliability. We argue this coverage–efficiency trade-off—made explicitly tunable by the coverage gate—is the honest characterization of when an agentic collector earns its cost.**

***Index Terms*—LLM security, threat intelligence, autonomous agents, tool use, multi-armed bandits, cyber threat intelligence, agentic systems**

I. Introduction

Large language models are now embedded in production software as chat assistants, retrieval-augmented applications, and tool-using agents. This deployment surface has produced a distinct class of vulnerabilities—prompt injection, jailbreaks, retrieval-augmented generation (RAG) poisoning, agent tool abuse, AI data leakage, and model-supply-chain compromise—codified in community taxonomies such as the OWASP Top 10 for LLM Applications [1] and demonstrated in the wild [2]. Unlike a single CVE feed, the evidence for these threats is fragmented: a newly exploited deserialization bug in a model server appears first in NVD [3] and the CISA KEV catalog [4]; an affected Python package surfaces in OSV.dev [5]; and the attack technique itself is often described weeks earlier in an arXiv preprint [6]. An effective LLM-security intelligence pipeline must therefore continuously harvest, cross-reference, and prioritize across all of these.

Automating this harvest is more than a crawling problem. Each source has its own retrieval semantics and a rich set of advanced operators—NVD exposes CWE filtering, CVSS-severity thresholds, publication-date windows, and a KEV flag; arXiv accepts a boolean field-scoped query language with category restrictions; OSV is package-centric (ecosystem, package name, purl); CISA KEV matches product and vulnerability-type tokens. The collection task is a sequence of decisions: *which sources to query this round*, *what query text and operators to use*, and *whether the marginal yield justifies another round*. In most pipelines, including the deterministic baseline we build upon, these decisions are made by hand-tuned rules: a fixed action sequence, template queries built from a topic→terms table, a round-robin over operators, and a stop condition driven by a coverage threshold. The signals needed to decide better—per-round novelty, duplicate ratio, noise ratio, per-source yield, open coverage gaps—are already recorded, but they are never shown to a reasoner.

This paper presents the architecture of an autonomous collection agent that closes that gap. The agent runs inside a native tool-use loop provided by the Claude Agent SDK [7] and takes over four decisions—source selection, query formulation, advanced-operator use, and termination—grounding each in a compact digest of the recorded metrics. Crucially, the agent does not replace the controller. A Python controller

keeps the round skeleton, enforces budgets through hooks, and owns persistence; every agent decision is a structured tool-forced call validated against a Pydantic schema, and any failure—timeout, parse error, validation error, or runtime unavailability—falls back to the *same* deterministic rule the baseline engine uses. The result is an engine whose floor is a verified rule-based system and whose ceiling is autonomous, multi-step reasoning.

Contributions. We make the following architectural contributions:

- A *hybrid controller* pattern (Section III–IV) in which an agentic engine and a deterministic engine share one rule module, selected by an engine factory, with a three-layer fallback (decision / round / engine) that renders every degradation observable rather than silent.
- A *typed source-tool layer* (Section V) that wraps four live sources as in-process MCP tools whose typed parameters expose advanced operators directly to the agent, with descriptions that teach effective operator combinations.
- A *bandit-informed source-selection* mechanism (Section VI) and a *three-tier relevance filter* (Section VII) that together turn raw collection counts into a trustworthy yield signal and a per-topic coverage measurement, plus a context-engineering discipline (Section VIII) that feeds summaries, never raw documents, to the decision loop.
- A *coverage-first termination policy* (Section IV) that makes completeness a hard constraint: a depth-quota coverage gate forbids stopping while a target topic is undercovered, a marginal-information stall detector prevents futile loops on unreachable topics, and both bound—rather than replace—the LLM stop decision under a strict priority order.
- A completed nine-goal *same-budget A/B evaluation* (Section IX) that makes the "intelligence gain" falsifiable, reporting coverage completeness as the primary metric alongside efficiency, volume, and reliability.

The emphasis of this paper is architecture, but we back it with a completed same-budget A/B study (Section IX) over nine frozen goals, two of them multi-topic. We report the outcome without cherry-picking: where the deterministic baseline wins on per-call efficiency, we say so, and we trace the agentic engine's coverage advantage to the specific mechanisms that produce it.

II. Background and Related Work

Cyber threat intelligence (CTI) automation. Structured CTI exchange formats such as STIX [8] and adversary-behavior knowledge bases such as MITRE ATT&CK [9] standardize *what* intelligence looks like once collected, but say little about the active, multi-source *acquisition* loop. Recent work applies LLMs to CTI knowledge-graph construction under data scarcity [10]; our system consumes such extractors downstream but the collection engine described here is upstream and independent of them.

LLM agents and tool use. The ReAct paradigm [11] interleaves reasoning traces with tool actions, and Toolformer [12] shows models can learn to call APIs; chain-of-thought prompting [13] underpins the multi-step decisions we elicit. The Model Context Protocol (MCP) [14] standardizes how tools are exposed to an agent, and the Claude Agent SDK [7] provides a native tool loop, in-process MCP servers, and lifecycle hooks. We use these mechanisms not as an end-to-end autonomous crawler but as the substrate for a tightly bounded set of decisions.

Source selection as a bandit problem. Choosing which feed to query under a fixed budget is naturally an exploration–exploitation problem. We adopt UCB1 [15], whose finite-time regret guarantees and parameter-free form suit a cold-start setting with sparse per-source reward history [16]. Unlike a pure bandit controller, we treat the bandit as an *advisor*: its ranking is rendered into the agent's context and may be overridden with a rationale.

Relevance filtering. Tiered filtering—cheap lexical rules first, dense-embedding similarity [17] next, an LLM judge last—lets us spend model calls only on genuinely ambiguous items. This "cheapest-first" cascade is the relevance analogue of staged retrieval and keeps the noise signal that drives both bandit reward and termination trustworthy.

III. System Overview

A. Design principle: a hybrid controller

The central architectural decision is that the LLM does *not* own the loop. Figure 1 shows the layered design. A Python controller owns four things the agent must never be trusted with: the round skeleton, the API-call budget and its safety valves, blackboard accounting, and persistence. The agent owns one thing: the decisions that benefit from reasoning over context. The two engines—a deterministic *rules* engine (`RealIntelRunController`) and an agentic *sdk* engine (`SdkIntelRunController`)—are interchangeable behind a factory:

> `create_intel_controller(...)` reads `INTEL_ENGINE` $\in$ {`rules`, `sdk`}, probes runtime availability, and returns the agentic controller only when both the SDK package and a provider key are present; otherwise it transparently returns the rules controller.

This factory is the outermost of three fallback layers (Section VIII). Because both engines emit the identical blackboard schema, run-file layout, and `real-` run-id prefix, every downstream consumer—the web console, the knowledge-graph adapter, and the evaluation harness—is engine-agnostic.

B. The blackboard and the round skeleton

State lives on a single `IntelRunBlackboard`: approved sources, accumulated `raw_items`, a `query_history` (one entry per round recording `novelty_score`, `noise_ratio`, `duplicate_ratio`, and per-source plans), `coverage_gaps`, `reflection_notes`, a `RunBudget` (max rounds, max API calls, max sources), and run `metrics`. The controller iterates rounds up to `max_rounds`; within each round it (1) renders

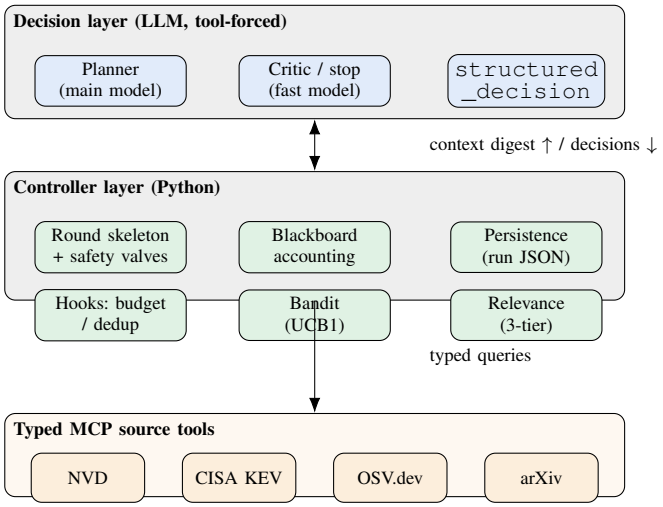

Fig. 1. Layered hybrid architecture. The LLM decision layer reasons over a compact context digest and returns tool-forced structured decisions; the Python controller layer owns the loop, budget hooks, bandit, relevance filter, accounting, and persistence; the four live sources are exposed as typed in-process MCP tools.

a context digest, (2) selects sources, (3) plans source-specific queries, (4) enforces the call budget, (5) executes queries and merges the batch (deduplicating against prior items and computing the per-round metrics), (6) runs relevance annotation and coverage analysis, and (7) decides whether to continue. Steps 2, 3, and 7 are the agentic decision points; the rest are deterministic bookkeeping shared verbatim with the rules engine via `intel/rules.py`.

IV. Autonomous Decision Architecture

A. From seven hard-coded decisions to four reasoned ones

The deterministic baseline contains seven decision points (action selection, yield assessment, coverage analysis, semantic expansion, strategy rewrite, completeness, and source-query construction), each implemented as a pure function in `intel/rules.py`. The agentic engine elevates four of these—the ones where reasoning over context pays off—to LLM decisions, and keeps the rest as deterministic post-processing. Table I maps each autonomous capability to its structured output schema and its rule fallback.

B. The structured-decision engine

Every decision flows through one mechanism, `StructuredDecisionEngine.decide`, which preserves the baseline's contract—"agent output preferred, deterministic rule as backstop"—while making failures countable. Given a decision name, a prompt, and a Pydantic output model, it:

1) converts the Pydantic model to a JSON schema (inlining `$defs` so nested proposal lists remain valid MCP tool inputs);
2) issues a *tool-forced* query: a single ephemeral MCP tool `submit_decision` is registered with that schema, and the prompt instructs the model to call it exactly once. Tool forcing proved more reliable across Anthropic-compatible endpoints than native structured-output modes (Section IX);
3) captures the tool input, or, defensively, parses raw JSON from the assistant text if a model answers in prose despite the instruction;
4) validates against the Pydantic model. On *any* exception—runtime unavailable, timeout, error result, parse failure, validation failure—it returns `None`, records the failure in telemetry, and the caller runs the rule.

TABLE I
THE FOUR AUTONOMOUS DECISIONS, THEIR STRUCTURED OUTPUT SCHEMAS, AND THE DETERMINISTIC FALLBACK INVOKED ON ANY FAILURE.

Capability	Output schema	Deterministic fallback
Source selection	`SourceSelectionDecision`	bandit top-k ranking executed directly
Query & operators	`CollectionPlanDecision` (per-source proposals)	`build_source_query_specs` (template + rotation)
Query rewrite	`RewriteDecisionOutput`	`fallback_rewrite_decision` (gap-driven)
Termination	`CompletenessDecisionOutput`	coverage / yield / budget rule

Because the engine is synchronous from the controller's perspective but the SDK is async, the call is bridged through a bounded event-loop runner that also works when an outer loop is already running (a thread-isolated `asyncio.run` with a hard timeout). A *test seam*—an injectable async `runner`—lets the entire decision path be unit-tested deterministically with a fake runner that consumes no API quota.

C. Source selection (bandit recommendation + agent veto)

Each round, the controller renders the bandit ranking for the current topic bucket into the digest and asks the agent to select 1–4 sources, "following the bandit ranking unless the digest gives a concrete reason to deviate." A valid, non-empty selection restricted to enabled sources is honored and logged with the agent's rationale and a `follow_bandit` flag for later analysis; an empty or invalid selection falls back to the bandit top-k. Thus the bandit guarantees a sensible floor even with no LLM, while the agent can explore an under-covered gap that only one source can fill.

D. Query formulation and advanced operators

The collection-plan decision is where the agent escapes templates. It receives the selected sources, the current mission query, and a per-source operator guide, and returns 2–6 proposals, each with a `source_name`, free-form `query_text`, a `params` dictionary, and a rationale. The controller defends this with a strict allow-list: only whitelisted parameter keys per source survive, empty values are dropped, proposals for unselected sources are discarded, and any proposal whose canonical key duplicates an already-executed query is removed. Surviving proposals become `SourceQuerySpec`s

Algorithm 1 One collection round (agentic engine)

$d \leftarrow$ RENDERDIGEST(blackboard, bandit) ▷ $\leq$ 2–3K tokens
$S \leftarrow$ DECIDESOURCES(d) ▷ schema: `SourceSelectionDecision`
if $S = \perp$ **then** $S \leftarrow$ BANDITTOPK()
end if ▷ fallback
$P \leftarrow$ DECIDECOLLECTIONPLAN(d, S) ▷ free queries + operators
if $P = \perp$ **then** $P \leftarrow$ BUILDSPECSBYRULE()
end if
$P \leftarrow$ ENFORCECALLBUDGET(P) ▷ controller-owned valve
$B \leftarrow$ EXECUTESPECS(P); MERGEINTOBLACKBOARD(B)
BANDITUPDATE(B); RELEVANCEANNOTATE(B_{new})
$g \leftarrow$ COVERAGEANALYSIS() ▷ deterministic
$r \leftarrow$ DECIDEREWRITE(d', g); **if** $r = \perp$ **then** rule rewrite
$c \leftarrow$ DECIDETERMINATION(d', INFOFEATURES) ▷ fast model
apply valves (priority $\max \succ$ stall $\succ$ cov $\succ \min \succ$ agent):
  if round$+1 \geq$ `max_rounds` **then** stop
  elif STALLED($g_{t-p..t}, \tau$) **then** stop ▷ exhausted
  elif OPENGAPS(q) $\wedge$ budget **then** continue ▷ cover
  elif round$+1 <$ `min_rounds` **then** continue
return (c.continue, r.nextQuery **or** GAPQUERY)

with `strategy_name = sdk_agent_proposal`; if none survive validation, the round degrades to the rule-built specs. This is parameter freedom *bounded by* code-side validation—the agent writes queries, but cannot issue malformed or repeated ones.

E. Coverage-first termination

When to stop is the decision with the most leverage over the system's stated goal: *cover every target topic* while tolerating noise. We therefore make the LLM stop decision the innermost of a five-level priority ladder, three levels of which are deterministic controller valves.

Depth-quota coverage. A target topic counts as covered only once at least q *relevant* items map to it (we use $q = 3$); a single weak hit does not suffice. Each item's topic is assigned by the embedding tier of the relevance filter (Section VII)—the argmax over per-topic anchor similarities—falling back to keyword detection when no embedder is configured. `coverage_completeness` is then the fraction of target topics at or above quota, and the *open gaps* are those below it.

Marginal-information signal. The "new information" a round actually buys is captured by a single scalar: $g_t = n_t^{\text{rel}}/c_t$, the number of items in round t that are simultaneously *new* (not a duplicate of any prior item) and *relevant*, divided by that round's API calls c_t. Because duplicates are excluded from n_t^{rel} and off-topic items from its relevance test, g_t penalizes exactly the two pathologies the operator cares about—high duplicate rate and high noise share—in one number. A search is *stalled* when g_t stays below a floor τ for the last p rounds (defaults $\tau = 0.5$, $p = 2$).

The priority ladder. On each round the controller resolves termination as

`max_rounds` $\succ$ `stall` $\succ$ `coverage gate` $\succ$ `min_rounds` $\succ$ `agen`

The `max_rounds` cap and API budget bound cost absolutely. Below them, the *stall* detector stops a run whose marginal information has collapsed—this is what prevents the coverage gate from burning the whole budget on a genuinely unreachable sparse topic, and when it fires with gaps still open it records them honestly. Below stall, the *coverage gate* forbids stopping while any target topic is under quota, overriding both the `min_rounds` floor and the agent: completeness is a hard constraint, not a suggestion. Only when coverage is met and the search is productive does the agent's own stop decision—returned on the fast model from a feature vector of recent g_t, novelty, duplicate and noise ratios, remaining budget, and open gaps—take effect. The same marginal-information features are rendered into the agent's context digest, so its reasoning and the deterministic valves read the same signal. If the gate forces another round but the agent proposed no query, the controller synthesizes a gap-targeted query for the open topics; if none can be formed, it stops. Autonomy thus operates strictly inside an envelope whose floor (coverage) and ceiling (budget) the operator sets.

V. Typed Source Tools and Advanced Operators

The four sources are exposed through an in-process MCP server, `intel_sources`, with one typed tool each. Each tool is a thin wrapper over the existing HTTP/parsing layer (zero new request code), so the agentic and rules engines hit identical source logic. The architectural value is in the *type signatures and descriptions*, which surface advanced operators the baseline could only reach through hand-written rotation:

- `search_nvd(keyword_search, keyword_exact_match, cwe_id, cvss_v3_severity, pub_start_date, pub_end_date, has_kev, cve_id)` — the description teaches combinations such as "product keyword + `cwe_id` (e.g. MLflow + CWE-502) targets AI supply-chain bugs" and "keyword + `has_kev` surfaces actively exploited issues."
- `search_arxiv(search_query)` — full arXiv query language: field prefixes (`all:`, `ti:`, `abs:`), boolean operators, quoted phrases, and category filters (`cat:cs.CR`).
- `search_cisa_kev(keyword, cve_ids)` — confirmed in-the-wild exploitation, intended to be used *after* NVD to check whether a class is actively exploited.
- `search_osv(ecosystem, package_name, version, purl, vuln_id)` — package-centric queries for the AI stack (model servers, agent frameworks, vector databases).

A `SourceCallRecorder` keeps the full `RawIntelItem` objects on the Python side while returning only a compact JSON digest (ids, truncated titles, relevance scores, topics) to the agent, so the controller's blackboard accounting stays authoritative and the agent's context stays small. In the current hybrid mode the controller executes the validated proposals; the same MCP server and hooks are already wired for a fully agentic mode in which the agent calls these tools directly inside the loop, gated on evaluation evidence.

VI. Bandit-Informed Source Selection

We model source selection as a UCB1 bandit [15] whose arms are `(source, topic_bucket)` pairs and whose reward is new deduplicated items per API call—a quantity derived entirely from existing blackboard records (per-source call counts from query plans, per-source new items from item-id prefixes). For an arm with empirical mean reward $\bar{x}_i$ and n_i pulls out of N total, the score is

$$\mathrm{UCB}_i = \bar{x}_i + c\sqrt{\frac{\ln \max(e, N)}{n_i}}, \tag{1}$$

with exploration constant $c = 1.4$; unpulled arms take score $+\infty$ (optimistic initialization) so every source is tried before any is exploited. Rewards are capped to bound the influence of a single lucky round, and per-topic statistics are merged with a `general` bucket so a cold topic still benefits from a source's overall track record. State persists in `data/bandit_state.json` and accumulates across runs.

Critically the bandit only *recommends*: its ranking and confidence are rendered into the agent context, and the agent's selection is recorded against the ranking. We validate the bandit independently of the LLM by offline replay (Section IX), comparing the realized reward of its top-2 recommendations against round-robin polling and a hindsight-best oracle—without spending any API quota.

VII. Three-Tier Relevance Filtering

Raw collection counts are a poor yield signal because feeds return many off-topic items. We replace the baseline's single keyword score with a cheapest-first cascade that spends model calls only where they change the verdict:

1) **Rule (zero cost).** The existing keyword relevance score is reused: items ≥ 0.75 are accepted and < 0.20 rejected outright; only the middle band advances.
2) **Embedding.** Middle-band items are scored by maximum cosine similarity between their title+summary and a set of per-topic anchor sentences (three hand-written exemplars per threat class). Scores are cached as JSONL keyed by a content hash, so an item is never embedded twice across runs; high/low thresholds resolve most items here.
3) **Flash LLM.** Only the residual uncertain band is adjudicated by a fast model in batches that return a JSON array of verdicts; on any failure the item keeps its tier-2 result.

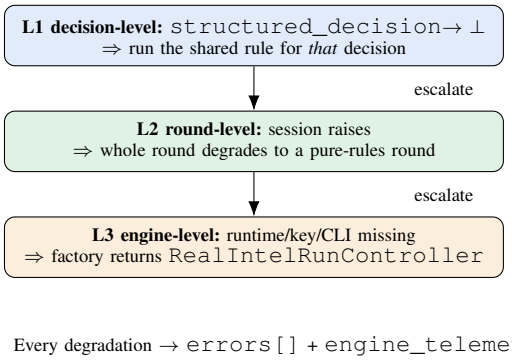

Fig. 2. Three-layer fallback. Each layer degrades to the shared deterministic rules at a coarser granularity, and—unlike the prior design—records the cause and layer so degradation is observable, not silent.

Each item records `relevance = {score, label, method, topic}`; the `method` field (rule / embedding / llm) makes per-tier precision measurable, and `topic`—the argmax target topic from the embedding tier's per-topic anchor similarities—is the input to the depth-quota coverage measurement that drives termination (Section IV-E). Improved relevance tightens the noise ratio, which in turn sharpens the bandit reward, the marginal-information signal, and the coverage gate—a closed signal-quality loop. The pipeline's network clients are injectable, so the entire cascade runs offline in tests, and it self-degrades to tier 1 only when no embedding key is configured.

VIII. Context Engineering and Reliability

A. Feed summaries, never raw documents

The single largest lever for cost and reliability without model training is context discipline. The only dynamic context a decision session sees is a digest of at most ∼2–3K tokens rendered by `render_round_context`: the run goal; a round/budget header; the last few queries with their new/total, duplicate, and noise metrics; a per-source totals table; the bandit ranking; open coverage gaps with ROI; the last reflection note; and a small sample of *newest item titles only*. Full `raw_items` are never dumped—the baseline's habit of inlining all items into a coverage prompt was its main source of context overflow and silent parse failure. Because the digest is the entire decision input and is persisted with the run, every decision is reproducible and auditable after the fact.

B. Three-layer fallback, never silent

Reliability rests on three nested fallbacks, shown in Figure 2, all degrading to the *same* `intel/rules.py` implementation so the two engines cannot drift:

- **L1 (decision):** a failed structured decision returns `None`; the caller runs the rule for that one decision and the round continues.
- **L2 (round):** an exception anywhere in a round is caught and the round re-runs as a pure-rules round; the run does not abort.

- **L3 (engine):** if the SDK runtime, provider key, or bundled CLI is unavailable, the factory never constructs the agentic engine at all.

The key departure from the baseline is observability: every fallback appends an `ErrorRecord` and updates `engine_telemetry` with the decision name, layer, and cause, so the structured-decision success rate and per-layer fallback rates become first-class evaluation metrics rather than invisible degradations.

C. Budget and de-duplication hooks

Budget enforcement lives in the local Python process, not the endpoint, via SDK `PreToolUse`/`PostToolUse` hooks. `PreToolUse` denies a source call when the run or per-round call cap is reached, or when the exact query was already executed, returning an instruction to the agent to change the query rather than failing silently; `PostToolUse` increments counters and records the call for blackboard accounting and bandit updates. Because hooks run locally they are the most reliable control point and do not depend on any endpoint capability.

D. Provider portability

The runtime targets Anthropic-*compatible* endpoints rather than a single hosted model. A provider abstraction selects DeepSeek (`deepseek-v4-pro`/`-flash`) or GLM by environment variable, with an explicit `ANTHROPIC_BASE_URL`/`AUTH_TOKEN` override, and maps a main and a fast model tier used respectively for planning and for the cheaper termination/relevance calls. All SDK imports are lazy so the package—and the rules engine—function without the agent dependency installed.

IX. Evaluation

We designed the evaluation to make "the agent is more intelligent" a falsifiable, same-budget claim rather than a qualitative impression.

A. Protocol

Nine collection goals are frozen as a benchmark set: seven single-topic or broad goals—*prompt-injection* (PI), *jailbreak* (JB), *RAG poisoning* (RAG), *agent tool abuse* (ATA), *model supply chain* (MSC), *data leakage* (DL), and a six-topic *broad LLM-security* sweep (BROAD)—plus two *multi-topic* goals added to make coverage non-trivial to satisfy: *IJA* (injection + jailbreak + agent-tool-abuse) and *SRD* (supply-chain + RAG + data-leakage). For each goal we run the `rules` and `sdk` engines under identical safety valves (max API calls, max rounds) and repeat each three times on live sources. The *primary* metric is `coverage_completeness`—the fraction of target topics reaching the depth quota $q = 3$ (Section IV-E)—computed engine-agnostically from the persisted blackboard. Secondary metrics are *efficiency* (deduplicated relevant items per API call), the per-round *marginal information* g_t, *volume* (items per call), *termination quality* (deviation between the actual stop round and the hindsight-optimal stop, the last round whose new-item yield exceeds 20% of the run's peak), *reliability* (structured-decision success and fallback rates), and *cost* (wall-clock). An item counts as relevant if its three-tier verdict is *relevant* or its rule score ≥ 0.68. To keep the three repeats independent, each `sdk` run is given a cold-start bandit so persistent state does not leak across repeats.

B. Coverage is the headline; efficiency is the price

Table II is the full result, and the primary metric tells a clear story (Figure 3a). On depth-quota coverage completeness the agentic engine reaches 1.00 on six of nine goals against four for the baseline, is *never* lower on any goal, and lifts the mean from 0.69 to 0.95 ($+38\%$). The gap is widest exactly where it matters: on agent-tool-abuse the deterministic templates reach the sparse target topic to quota *never* (0.00) while the agent reaches it every time (1.00); on RAG, $0.50 \rightarrow 1.00$; and on the two multi-topic goals, $0.67 \rightarrow 0.89$ (IJA) and $0.33 \rightarrow 0.78$ (SRD).

That completeness has a price, and we report it plainly (Figure 3b). On raw relevant-items-per-call the agent wins on PI, JB, and DL (the last by $+109\%$), but *loses* on the goals where the coverage gate forces it to keep chasing a sparse topic (RAG -26%, ATA -39%, MSC -17%, BROAD/IJA/SRD -22 to -26%); averaged over goals it is 8% *lower*. The mechanism is visible in the same table: the agent retrieves 60% more items per call and runs more rounds, because reaching a rare topic means issuing more, broader queries whose marginal items are individually less likely to clear the relevance bar. Figure 4 makes the trade explicit: under the agent every goal moves *rightward* toward full coverage, and the per-call efficiency moves up or down depending on how sparse the remaining topics are.

C. Why per-call efficiency is the wrong sole yardstick

The agent-tool-abuse goal is the cleanest illustration. There the deterministic baseline posts one of the highest efficiencies in the study (3.22 rel./call)—yet its coverage completeness is 0.00: its fixed keyword templates and round-robin never surface that sparse class to quota, so it efficiently re-collects easier topics and exhausts its rounds without ever covering the goal's namesake. The agent's 1.96 rel./call looks worse only because it actually spends calls reaching the rare topic, which it covers fully (1.00). The baseline's apparent efficiency lead is thus an artifact of *efficiently collecting the wrong thing*. The same pattern holds on the multi-topic goals SRD ($0.33 \rightarrow 0.78$) and IJA ($0.67 \rightarrow 0.89$) and on BROAD ($0.67 \rightarrow 0.89$): the coverage gate keeps the agent working the under-covered topic until quota or budget, which is exactly the behavior the stated objective demands. Across all goals the agent covers $+34\%$ more distinct topics and retrieves $+60\%$ more items per call (Table II), the latter a direct consequence of its heavier, agent-chosen use of advanced operators.

D. The marginal-information signal and cheap adjudication

The three-tier cascade keeps relevance annotation nearly free: across all agentic runs the embedding tier adjudicated

TABLE II

SAME-BUDGET A/B RESULTS ACROSS NINE FROZEN GOALS (MEAN OF THREE RUNS EACH; R = DETERMINISTIC `RULES`, S = AGENTIC `SDK`). **COVERAGE COMPLETENESS** (DEPTH QUOTA $q = 3$) IS THE PRIMARY METRIC. THE `SDK` ENGINE IS NEVER BELOW THE BASELINE ON COVERAGE AND FAR ABOVE IT ON THE HARD AND MULTI-TOPIC GOALS, TRADING PER-CALL EFFICIENCY FOR THAT COMPLETENESS. BOLD MARKS THE BETTER ENGINE WHERE THE GAP IS MATERIAL.

	Coverage		**Relevant/call**		**Items/call**		**Topics**		**Rounds**	
Goal	R	S	R	S	R	S	R	S	R	S
PI	1.00	1.00	2.47	**2.83**	3.16	**4.34**	2.0	**4.7**	1.0	5.0
JB	1.00	1.00	2.53	**3.48**	3.16	**6.41**	3.0	**4.0**	1.0	5.3
RAG	0.50	**1.00**	**2.53**	1.88	3.16	**4.75**	3.0	**5.0**	1.0	4.7
ATA	0.00	**1.00**	**3.22**	1.96	4.15	**6.62**	4.0	**5.3**	5.0	3.7
MSC	1.00	1.00	**2.32**	1.93	3.16	**5.73**	**4.0**	3.0	1.0	3.3
DL	1.00	1.00	1.53	**3.20**	3.16	**7.76**	2.0	**4.3**	1.0	2.7
BROAD	0.67	**0.89**	**3.66**	2.72	5.33	**8.60**	5.0	**6.0**	8.0	7.7
IJA	0.67	**0.89**	**4.27**	3.16	6.02	5.87	4.0	4.3	8.0	5.0
SRD	0.33	**0.78**	**3.21**	2.51	4.21	**6.71**	4.0	**5.0**	1.0	4.7
Mean	0.69	**0.95**	**2.86**	2.63	3.95	**6.31**	3.4	**4.6**	–	–

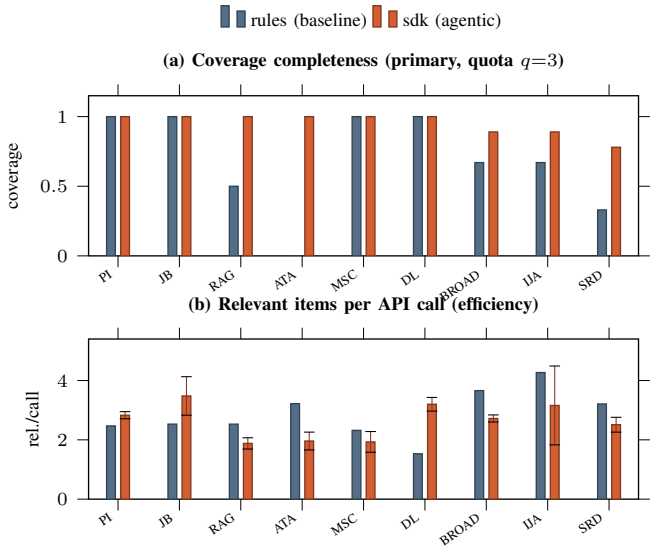

Fig. 3. Same-budget comparison per goal. (a) Coverage completeness, the primary objective: the agent matches or exceeds the baseline everywhere and closes the gaps the fixed policy cannot (ATA 0.00 → 1.00). (b) Per-call efficiency is the price paid for that coverage—higher on PI/JB/DL, lower where sparse topics must be chased. Error bars: ±1 s.d. over three runs (the deterministic baseline is exactly reproducible, zero variance).

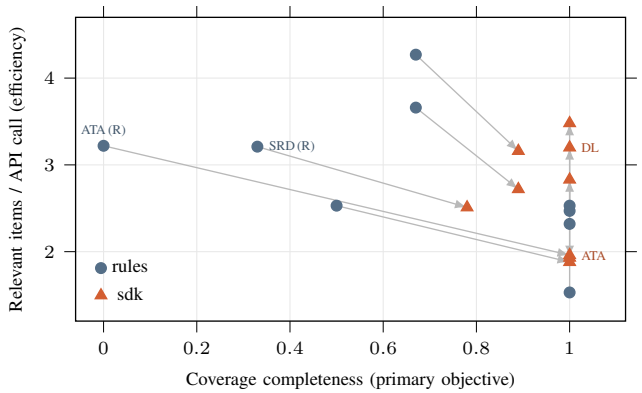

Fig. 4. Coverage–efficiency trade-off. Each arrow runs from a goal's deterministic point to its agentic point. Every goal moves *rightward* to higher coverage (the sparse-topic goals ATA and SRD travel furthest, from 0.0/0.33 to 1.0/0.78); per-call efficiency rises or falls depending on how sparse the topics it must still reach are.

3,256 items while the costly flash-LLM tier was invoked on only 70 ($\approx$ 1%), the remainder resolved by the zero-cost rule tier. Those same labels feed the per-round marginal-information signal g_t that the termination ladder watches. Figure 5 plots g_t over rounds for three representative runs: it starts high and decays unevenly as a goal's easy items are exhausted, with transient rebounds when the agent reformulates onto a fresh sub-topic. The stall floor $\tau = 0.5$ is drawn dashed; in this benchmark no run's g_t stayed below it for the two-round patience window, so the stall valve never fired (Section IX-E) and runs ended by coverage completion or the round cap first. The signal is nonetheless the safety mechanism that bounds the coverage gate when a sparse topic is genuinely unreachable.

E. Termination, reliability, and cost

On *termination quality* the agent is cleanly better: its stop round matched the hindsight-optimal round on all nine goals (mean deviation 0.0), whereas the deterministic stop condition overshot (mean deviation 0.22, e.g. a round late on BROAD)—its rule cannot read a yield plateau. The stall valve was available on every run but never triggered (stall rate 0 on all goals): the marginal-information signal never stayed below τ for the patience window before either coverage completed or the round cap intervened, so the valve served purely as the latent guard it is designed to be. On *reliability*, across 436 live structured decisions the engine recorded a 97.7% success rate (426/436) with a 2.3% fallback rate, every fallback logged with its layer and cause; no run aborted. This confirms the endpoint capability spike, which had recorded 100% tool-call (20/20) and structured-decision parse (20/20) reliability on the

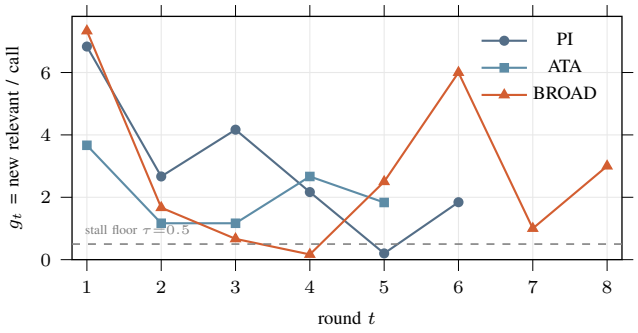

Fig. 5. The marginal-information signal g_t (new relevant items per API call) per round for three representative agentic runs. It decays unevenly with reformulation rebounds; the stall valve stops a run only if g_t stays below τ for the patience window, which never occurs here—coverage or the round cap ends runs first.

DeepSeek Anthropic-compatible endpoint, with native JSON-schema output and prompt caching unsupported—precisely why the architecture defaults to tool forcing and summary-based cost control. The *cost* is the honest counterweight: agentic runs took roughly an order of magnitude more wall-clock (∼20–112 s for `rules`; ∼200–1950 s for `sdk`), the price of an LLM in the loop, more rounds, and multi-round tool use.

Independently, the UCB1 source recommender was validated offline by replaying 33 historical runs (617 rounds): bandit regret 1.063 versus round-robin regret 1.245 (lower is better), confirming the recommender beats uniform polling with no API spend.

F. Threats to validity

Three caveats bound these claims. (i) *Relevance is heuristic*: the per-call "relevant" counts and the coverage assignment rest on the three-tier label, not human gold; a precision audit with two annotators and a cross-family LLM judge is the next step, and would firm up the efficiency comparison most. (ii) *Variance*: making completeness a hard constraint sharply reduced the agent's run-to-run spread relative to a yield-only stop—coverage standard deviation is 0 on six of nine goals and ≤ 0.19 elsewhere, and efficiency s.d. is ≤ 0.35 rel./call on seven goals, with IJA the lone outlier (± 1.33); the deterministic baseline remains exactly reproducible. (iii) *Endpoint specificity*: results are for one provider (DeepSeek); the portability layer is in place but the GLM path is unmeasured here.

X. Discussion, Limitations, and Future Work

When does autonomy pay? The evaluation gives a concrete answer rather than a slogan. Autonomy pays when completeness matters and the goal contains a sparse or hard-to-phrase topic that fixed templates miss (ATA, SRD, IJA, BROAD): there the coverage gate plus the agent's freedom to reformulate queries close gaps the baseline cannot (+38% mean coverage; ATA $0.00 \rightarrow 1.00$), and coverage—not per-call efficiency—is the metric that moves. Autonomy is *not* worth its cost when a goal is narrow, already reachable by a hand-tuned template, and judged only on per-call efficiency (RAG, MSC): the deterministic engine matches the agent's coverage at a fraction of the wall-clock. A practical deployment can therefore route by goal breadth, reserving the agentic engine for broad or sparse missions and the cheap deterministic engine for narrow ones—a routing the shared engine factory already makes a one-line choice.

Why a hybrid, not a fully autonomous agent. A fully agentic loop—the agent calling source tools directly until it decides to stop—is technically unlocked: the MCP tools and hooks are in place and live tool reliability was 97.7%. We nonetheless keep the controller authoritative over budget, coverage, and persistence, because an architecture whose worst case is a verified rule engine is safer to deploy and easier to evaluate than one whose worst case is an unbounded model; the order-of-magnitude cost gap also argues for keeping the loop bounded. Full autonomy remains gated on evidence, not capability.

Limitations. (i) The relevance signal, though improved by the three-tier filter, remains heuristic, and it now also drives the coverage assignment; the precision audit (Section IX, threats) is the highest-value next step and would most affect both the efficiency *and* coverage comparisons. (ii) The bandit's reward (new items per call) optimizes *novelty*, which can under-weight a small number of high-value confirmations; a value-weighted reward is future work. (iii) The agentic engine adds an LLM in the loop, costing roughly an order of magnitude more wall-clock; context discipline and fast-model termination mitigate but do not eliminate this. (iv) Provider portability is validated mainly on one endpoint; the GLM path is retained as a fallback pending subscription renewal.

Future work. Beyond completing the A/B study, the natural next steps are a value-weighted bandit reward, a fully agentic mode behind the existing tools and hooks, and feeding the per-item relevance `method` labels back as supervision for the filter. The collected, relevance-annotated items are already the input corpus for a downstream knowledge-graph construction stage, which we treat as out of scope here.

XI. Conclusion

We described the architecture of an autonomous LLM-security intelligence-collection agent built on a hybrid controller: an LLM reasons over a compact, persisted metric digest to make four decisions—source selection, query and operator formulation, rewrite, and termination—while a Python controller retains the loop, budget safety valves, accounting, and persistence. Termination is governed by a coverage-first policy: a depth-quota coverage gate makes completeness a hard constraint and a marginal-information stall detector guards against futile loops, with the LLM stop decision operating only inside that envelope. A shared deterministic rule module serves simultaneously as the baseline engine and as the per-decision, per-round, and per-engine fallback, so the system's floor is a verified rule-based collector and its degradations are observable rather than silent. A same-budget A/B study over nine frozen goals characterizes the outcome honestly: on the

primary objective the agentic engine matches or exceeds the baseline's coverage on every goal and lifts the mean from 0.69 to 0.95 ($+38\%$), closing sparse gaps the fixed policy never reaches (agent-tool-abuse $0.00 \rightarrow 1.00$), while retrieving $+60\%$ more items per call and terminating at the hindsight-optimal round throughout, at 97.7% live decision reliability. The price is per-call efficiency—8% lower on average and an order of magnitude more wall-clock—paid precisely where the agent chases topics the baseline abandons. The contribution is thus not only an architecture but a clear account of *when* autonomy in intelligence collection earns its cost—when coverage completeness matters and topics are sparse—enabling a goal-aware choice between the two engines the design already unifies.

REFERENCES

[1] OWASP Foundation, "OWASP top 10 for large language model applications," https://owasp.org/www-project-top-10-for-large-language-model-applications/, 2025, accessed: 2026-06-13.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[3] National Institute of Standards and Technology, "National vulnerability database (NVD) CVE API 2.0," https://nvd.nist.gov/developers/vulnerabilities, 2024, accessed: 2026-06-13.

[4] Cybersecurity and Infrastructure Security Agency, "Known exploited vulnerabilities catalog," https://www.cisa.gov/known-exploited-vulnerabilities-catalog, 2024, accessed: 2026-06-13.

[5] Open Source Vulnerabilities, "OSV.dev: A distributed vulnerability database for open source," https://osv.dev/, 2024, accessed: 2026-06-13.

[6] arXiv, "arXiv API user manual," https://info.arxiv.org/help/api/index.html, 2024, accessed: 2026-06-13.

[7] Anthropic, "Claude agent SDK for Python," https://pypi.org/project/claude-agent-sdk/, 2026, version 0.2.99. Accessed: 2026-06-13.

[8] OASIS Cyber Threat Intelligence Technical Committee, "STIX version 2.1: Structured threat information expression," https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html, 2021, accessed: 2026-06-13.

[9] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," in *The MITRE Corporation, Technical Report*, 2018.

[10] Y. Cheng, O. Bajaber, S. A. Tsokkis, Y. Yao, K. Fu, L. Wang, Y. Zhang, and D. D. Yao, "CTINEXUS: Leveraging optimized LLM in-context learning for constructing cybersecurity knowledge graphs under data scarcity," *arXiv preprint arXiv:2410.21060*, 2024.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[12] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] Anthropic, "Introducing the model context protocol," https://www.anthropic.com/news/model-context-protocol, 2024, accessed: 2026-06-13.

[15] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.

[16] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.

[17] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.